AndroidManifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.EXP5">
        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="@string/app_name"
            android:theme="@style/Theme.EXP5">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

build.gradle.kts (Module: app)

```kotlin
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
}

android {
    namespace = "com.example.exp5"
    compileSdk = 36
    buildToolsVersion = "36.1.0"

    defaultConfig {
        applicationId = "com.example.exp5"
        minSdk = 31
        targetSdk = 36
        versionCode = 1
        versionName = "1.0"

        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
            isMinifyEnabled = false
            proguardFiles(
                getDefaultProguardFile("proguard-android-optimize.txt"),
                "proguard-rules.pro"
            )
```

```
        }
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures {
        compose = true
    }
}

dependencies {
    implementation(libs.mpandroidchart)
    implementation(platform(libs.androidx.compose.bom))
    implementation(libs.androidx.activity.compose)
    implementation(libs.androidx.compose.material3)
    implementation(libs.androidx.compose.ui)
    implementation(libs.androidx.compose.ui.graphics)
    implementation(libs.androidx.compose.ui.tooling.preview)
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.lifecycle.runtime.ktx)
    testImplementation(libs.junit)
    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.compose.ui.test.junit4)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
    debugImplementation(libs.androidx.compose.ui.test.manifest)
    debugImplementation(libs.androidx.compose.ui.tooling)
}
```

libs.versions.toml (Required Entries)

```
[versions]
agp = "9.2.0"
coreKtx = "1.18.0"
lifecycleRuntimeKtx = "2.10.0"
activityCompose = "1.13.0"
kotlin = "2.2.10"
composeBom = "2026.02.01"
mpandroidchart = "3.1.0"
[libraries]
mpandroidchart = { group = "com.github.PhilJay", name = "MPAndroidChart", version.ref =
    "mpandroidchart" }
androidx-core-ktx = { group = "androidx.core", name = "core-ktx", version.ref = "coreKtx" }
androidx-lifecycle-runtime-ktx = { group = "androidx.lifecycle", name = "lifecycle-runtime-ktx"
    , version.ref = "lifecycleRuntimeKtx" }
androidx-activity-compose = { group = "androidx.activity", name = "activity-compose",
    version.ref = "activityCompose" }
androidx-compose-bom = { group = "androidx.compose", name = "compose-bom", version.ref =
    "composeBom" }
androidx-compose-ui = { group = "androidx.compose.ui", name = "ui" }
androidx-compose-ui-graphics = { group = "androidx.compose.ui", name = "ui-graphics" }
androidx-compose-ui-tooling-preview = { group = "androidx.compose.ui", name =
    "ui-tooling-preview" }
androidx-compose-material3 = { group = "androidx.compose.material3", name = "material3" }
[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
```

MainActivity.kt

```
package com.example.exp5

import android.graphics.Color as AndroidColor
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.verticalScroll
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.*
import androidx.compose.runtime.Composable
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.compose.ui.viewinterop.AndroidView
import com.example.exp5.ui.theme.EXP5Theme
import com.github.mikephil.charting.charts.BarChart
import com.github.mikephil.charting.charts.LineChart
import com.github.mikephil.charting.charts.PieChart
import com.github.mikephil.charting.components.XAxis
import com.github.mikephil.charting.data.*
import com.github.mikephil.charting.formatter.IndexAxisValueFormatter

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            EXP5Theme {
                Scaffold(
                    modifier = Modifier.fillMaxSize(),
                    containerColor = Color(0xFF0F172A) // Deep Navy background
                ) { innerPadding ->
                    FinancialDashboard(
                        modifier = Modifier.padding(innerPadding)
                    )
                }
            }
        }
    }
}

@Composable
fun FinancialDashboard(modifier: Modifier = Modifier) {
    val scrollState = rememberScrollState()
    Column(
        modifier = modifier
            .fillMaxSize()
```

```
            .verticalScroll(scrollState)
            .padding(16.dp),
        horizontalAlignment = Alignment.Start
    ) {
        Text(
            text = "Financial Overview",
            color = Color.White,
            fontSize = 28.sp,
            fontWeight = FontWeight.Bold,
            modifier = Modifier.padding(bottom = 8.dp)
        )
        Text(
            text = "Your revenue performance over the last 6 months",
            color = Color.Gray,
            fontSize = 14.sp,
            modifier = Modifier.padding(bottom = 24.dp)
        )

        // Summary Cards
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(16.dp)
        ) {
            SummaryCard(
                title = "Total Balance",
                amount = "$45,231.89",
                trend = "+12.5%",
                modifier = Modifier.weight(1f),
                color = Color(0xFF3B82F6)
            )
            SummaryCard(
                title = "Monthly Profit",
                amount = "$8,432.00",
                trend = "+5.2%",
                modifier = Modifier.weight(1f),
                color = Color(0xFF10B981)
            )
        }

        Spacer(modifier = Modifier.height(24.dp))

        // Line Chart Section
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .height(350.dp),
            colors = CardDefaults.cardColors(containerColor = Color(0xFF1E293B)),
            shape = RoundedCornerShape(24.dp),
            elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = "Revenue Stream",
                    color = Color.White,
                    fontSize = 18.sp,
                    fontWeight = FontWeight.SemiBold,
                    modifier = Modifier.padding(bottom = 16.dp)
                )
```

```
                FinancialChart(modifier = Modifier.fillMaxSize())
            }
        }
        Spacer(modifier = Modifier.height(16.dp))

        // Bar Chart Section
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .height(350.dp),
            colors = CardDefaults.cardColors(containerColor = Color(0xFF1E293B)),
            shape = RoundedCornerShape(24.dp),
            elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = "Monthly Comparison",
                    color = Color.White,
                    fontSize = 18.sp,
                    fontWeight = FontWeight.SemiBold,
                    modifier = Modifier.padding(bottom = 16.dp)
                )

                FinancialBarChart(modifier = Modifier.fillMaxSize())
            }
        }

        Spacer(modifier = Modifier.height(16.dp))

        // Pie Chart Section
        Card(
            modifier = Modifier
                .fillMaxWidth()
                .height(350.dp),
            colors = CardDefaults.cardColors(containerColor = Color(0xFF1E293B)),
            shape = RoundedCornerShape(24.dp),
            elevation = CardDefaults.cardElevation(defaultElevation = 8.dp)
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = "Expense Categories",
                    color = Color.White,
                    fontSize = 18.sp,
                    fontWeight = FontWeight.SemiBold,
                    modifier = Modifier.padding(bottom = 16.dp)
                )

                FinancialPieChart(modifier = Modifier.fillMaxSize())
            }
        }

        Spacer(modifier = Modifier.height(32.dp)) // Extra padding at bottom
    }
}

@Composable
fun SummaryCard(
```

```
    title: String,
    amount: String,
    trend: String,
    color: Color,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier,
        colors = CardDefaults.cardColors(containerColor = Color(0xFF1E293B)),
        shape = RoundedCornerShape(16.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(text = title, color = Color.Gray, fontSize = 12.sp)
            Spacer(modifier = Modifier.height(4.dp))
            Text(text = amount, color = Color.White, fontSize = 20.sp, fontWeight =
    FontWeight.Bold)
            Spacer(modifier = Modifier.height(4.dp))
            Text(text = trend, color = color, fontSize = 12.sp, fontWeight = FontWeight.Medium)
        }
    }
}

@Composable
fun FinancialPieChart(modifier: Modifier = Modifier) {
    val entries = listOf(
        PieEntry(40f, "Housing"),
        PieEntry(25f, "Food"),
        PieEntry(15f, "Transport"),
        PieEntry(10f, "Entertainment"),
        PieEntry(10f, "Other")
    )

    val colors = listOf(
        AndroidColor.parseColor("#3B82F6"), // Blue
        AndroidColor.parseColor("#10B981"), // Green
        AndroidColor.parseColor("#F59E0B"), // Amber
        AndroidColor.parseColor("#EF4444"), // Red
        AndroidColor.parseColor("#8B5CF6")  // Purple
    )

    val dataSet = PieDataSet(entries, "Categories").apply {
        setColors(colors)
        valueTextColor = AndroidColor.WHITE
        valueTextSize = 14f
        sliceSpace = 3f
    }

    val pieData = PieData(dataSet)

    AndroidView(
        factory = { context ->
            PieChart(context).apply {
                data = pieData
                description.isEnabled = false
                legend.apply {
                    textColor = AndroidColor.WHITE
                    verticalAlignment =
    com.github.mikephil.charting.components.Legend.LegendVerticalAlignment.CENTER
```

```
                    horizontalAlignment =
    com.github.mikephil.charting.components.Legend.LegendHorizontalAlignment.RIGHT
                    orientation =
    com.github.mikephil.charting.components.Legend.LegendOrientation.VERTICAL
                    setDrawInside(false)
                }
                setHoleColor(AndroidColor.TRANSPARENT)
                setTransparentCircleColor(AndroidColor.TRANSPARENT)
                setEntryLabelColor(AndroidColor.WHITE)
                setEntryLabelTextSize(12f)
                animateY(1400)
            }
        },
        update = { chart ->
            chart.data = pieData
            chart.invalidate()
        },
        modifier = modifier
    )
}

@Composable
fun FinancialBarChart(modifier: Modifier = Modifier) {
    val entries = listOf(
        BarEntry(0f, 15000f),
        BarEntry(1f, 18000f),
        BarEntry(2f, 16000f),
        BarEntry(3f, 22000f),
        BarEntry(4f, 20000f),
        BarEntry(5f, 28000f)
    )

    val dataSet = BarDataSet(entries, "Revenue").apply {
        color = AndroidColor.parseColor("#10B981") // Green
        valueTextColor = AndroidColor.WHITE
        valueTextSize = 10f
        setDrawValues(true)
    }

    val barData = BarData(dataSet).apply {
        barWidth = 0.6f
    }

    AndroidView(
        factory = { context ->
            BarChart(context).apply {
                data = barData
                description.isEnabled = false
                legend.isEnabled = false

                xAxis.apply {
                    position = XAxis.XAxisPosition.BOTTOM
                    textColor = AndroidColor.WHITE
                    setDrawGridLines(false)
                    axisLineColor = AndroidColor.TRANSPARENT
                    granularity = 1f
                }
```

```
                axisLeft.apply {
                    textColor = AndroidColor.WHITE
                    setDrawGridLines(true)
                    gridColor = AndroidColor.parseColor("#334155")
                    axisLineColor = AndroidColor.TRANSPARENT
                }

                axisRight.isEnabled = false
                setTouchEnabled(true)
                setPinchZoom(true)
                animateY(1500)
            }
        },
        update = { chart ->
            chart.data = barData
            chart.invalidate()
        },
        modifier = modifier
    )
}

@Composable
fun FinancialChart(modifier: Modifier = Modifier) {
    val months = arrayOf("Jan", "Feb", "Mar", "Apr", "May", "Jun")
    val entries = listOf(
        Entry(0f, 12000f),
        Entry(1f, 15000f),
        Entry(2f, 13500f),
        Entry(3f, 19000f),
        Entry(4f, 17500f),
        Entry(5f, 24000f)
    )

    val dataSet = LineDataSet(entries, "Revenue").apply {
        mode = LineDataSet.Mode.CUBIC_BEZIER
        color = AndroidColor.parseColor("#3B82F6")
        setCircleColor(AndroidColor.parseColor("#3B82F6"))
        lineWidth = 3f
        circleRadius = 5f
        setDrawCircleHole(true)
        circleHoleColor = AndroidColor.parseColor("#1E293B")
        setDrawValues(false)
        setDrawFilled(true)
        fillDrawable = null // Could use gradient here if we had access to resources
        fillAlpha = 50
        fillColor = AndroidColor.parseColor("#3B82F6")
    }

    val lineData = LineData(dataSet)

    AndroidView(
        factory = { context ->
            LineChart(context).apply {
                data = lineData
                description.isEnabled = false
                legend.isEnabled = false

                xAxis.apply {
```

```
                    position = XAxis.XAxisPosition.BOTTOM
                    valueFormatter = IndexAxisValueFormatter(months)
                    textColor = AndroidColor.WHITE
                    setDrawGridLines(false)
                    axisLineColor = AndroidColor.TRANSPARENT
                    granularity = 1f
                }

                axisLeft.apply {
                    textColor = AndroidColor.WHITE
                    setDrawGridLines(true)
                    gridColor = AndroidColor.parseColor("#334155")
                    axisLineColor = AndroidColor.TRANSPARENT
                }

                axisRight.isEnabled = false
                setTouchEnabled(true)
                setPinchZoom(true)
                animateX(1500)
            }
        },
        update = { chart ->
            chart.data = lineData
            chart.invalidate()
        },
        modifier = modifier
    )
}
```

Color.kt

```
package com.example.exp5.ui.theme

import androidx.compose.ui.graphics.Color

val NavyDeep = Color(0xFF0F172A)
val NavyLight = Color(0xFF1E293B)
val BluePrimary = Color(0xFF3B82F6)
val GreenSuccess = Color(0xFF10B981)
val TextPrimary = Color(0xFFFFFFFF)
val TextSecondary = Color(0xFF94A3B8)
```

Theme.kt

```
package com.example.exp5.ui.theme

import android.app.Activity
import android.os.Build
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.LocalContext

private val DarkColorScheme = darkColorScheme(
```

```
    primary = BluePrimary,
    secondary = NavyLight,
    tertiary = GreenSuccess,
    background = NavyDeep,
    surface = NavyLight,
    onPrimary = TextPrimary,
    onSecondary = TextPrimary,
    onTertiary = TextPrimary,
    onBackground = TextPrimary,
    onSurface = TextPrimary
)

private val LightColorScheme = lightColorScheme(
    primary = BluePrimary,
    secondary = NavyLight,
    tertiary = GreenSuccess,
    background = Color.White,
    surface = Color.White,
    onPrimary = Color.White,
    onSecondary = Color.Black,
    onTertiary = Color.White,
    onBackground = Color.Black,
    onSurface = Color.Black
)

@Composable
fun EXP5Theme(
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = true,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColorScheme
        else -> LightColorScheme
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
```

Type.kt

```
package com.example.exp5.ui.theme

import androidx.compose.material3.Typography
import androidx.compose.ui.text.TextStyle
import androidx.compose.ui.text.font.FontFamily
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.sp
```

```kotlin
// Set of Material typography styles to start with
val Typography = Typography(
    bodyLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 16.sp,
        lineHeight = 24.sp,
        letterSpacing = 0.5.sp
    )
    /* Other default text styles to override
    titleLarge = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Normal,
        fontSize = 22.sp,
        lineHeight = 28.sp,
        letterSpacing = 0.sp
    ),
    labelSmall = TextStyle(
        fontFamily = FontFamily.Default,
        fontWeight = FontWeight.Medium,
        fontSize = 11.sp,
        lineHeight = 16.sp,
        letterSpacing = 0.5.sp
    )
    */
)
```

strings.xml

```xml
<resources>
    <string name="app_name">EXP 5</string>
</resources>
```

themes.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<resources>

    <style name="Theme.EXP5" parent="android:Theme.Material.Light.NoActionBar" />
</resources>
```